Lesson 2 - Hello World

Bryan A. *CrazyUncleBurton* Thompson
bryan@batee.com

March 9, 2026

Overview

This lesson will show you how to compile and upload a basic "Hello World" program to a microcontroller.

Theory

This is our first program and we're learning a lot of things all at once. We make the circuit as easy as possible by using a pre-built module which only connects one way to the microcontroller.

Hardware Required

The following hardware is needed for this lesson:

1-M5Stack Tab5 Microcontroller Development Board

1-M5Stack Dual LED Module

1-4 wire M5Stack sensor cable

1-USB cable to connect the Tab5 USB-C port to your PC

Objectives

- Familiarity with the Tab5 development board
- Familiarity with VS Code IDE
- Create our first electric circuit
- Compile our first program
- Upload a basic "Hello World" program to the development board
- Test the program

The M5Stack Tab5 Microcontroller Development Board

M5Stack.com is owned by Espressif, the manufacturer of the microcontroller inside the Tab5. We chose it for the ultimate in support by the manufacturer. The M5Stack Tab5 microcontroller development board has an ESP32-P4 microcontroller with three cores, two of which run at 360MHz.

It has an ESP32-C6 Communications IC with support for 2.4 GHz Wi-Fi 6, Bluetooth 5 (LE), Zigbee 3.0, and Thread 1.3. It has a 5" LCD touchscreen, camera, microSD slot, Audio Inputs and Outputs, Microphones, a Real Time Clock, a 6-axis accelerometer/gyro sensor, and many ways to connect to external circuits!

The manufacturers datasheet for the Tab5 is located here. You can find information like schematics, external hardware connections and more:
`https://docs.m5stack.com/en/core/Tab5`

M5Stack provides a super-rich development ecosystem, including sensors, modules and accessories:
`https://shop.m5stack.com/collections/unit`

Figure 1: The M5Stack Tab5 Development Board

Figure 2: Tab5 Bottom Side

1. We can supply power to the Tab5 through its USB-C port. Connect the Tab5 USB-C port to your computer with a USB cable.

2. The white "U" shaped button on the left is the On/Reset/Off button.

3. Press the button once to turn on the screen. You should see the factory firmware running on the controller. It shows some useful data about the hardware and some live data on the LCD:

Figure 3: Tab5 Running Factory Firmware

4. Press and release the power button once. Note that the unit powers off and back on (we call this a reset).

5. Double press and release the power button to turn the controller off.

The PORT.A Input / Output Connector

Figure 4: The PORT.A Input/Output Connector

The connector is used to quickly connect various sensors to the Tab5. It enables quick, plug-and-play connectivity for sensors and actuators, often powered by the 5V rail. PORT.A can be configured for GPIO (digital I/O), or UART serial communication as well as I2C serial communication. We call this Rapid Prototyping because the circuit can be built without building a circuit board.

This is a 4-pin HY2.0-4P Grove connector. It's close to a 4 pin JST PH connector.

HY2.0-4P	Black	Red	Yellow	White
PORT.A	GND	5V	G53	G54

Figure 5: PORT.A Pins and their Functions

There are four pins on the PORT.A connector:

- Black - GND - This is the common reference terminal against which other outputs are measured.
- Red - 5V - This is a 5V reference / power supply from the port, intended to power any sensors or actuators. Why is it 5V when we're working in a 3.3V ecosystem? My guess is to let us power and communicate with 5V sensors as well as 3.3V ones.
- Yellow - G53 - This is connected to pin G53 on the microcontroller. It can be turned on and off programmatically. When it's on, it will be +3.3V with respect to ground. When it's off, it will be 0V.
- White - G54 - This is connected to pin G54 on the microcontroller. It can be turned on and off programmatically. When it's on, it will be +3.3V with respect to ground. When it's off, it will be 0V.

PORT.A has some other special functions which we will cover in later lessons.

The Dual LED Module

For any Hello World program, we need an LED to blink. Unfortunately, there's not a blinkable LED built into our Tab5 Dev board (a huge oversight on the part of M5Stack). CrazyUncleBurton didn't let that stop him - he bought some prototype modules and built his own!

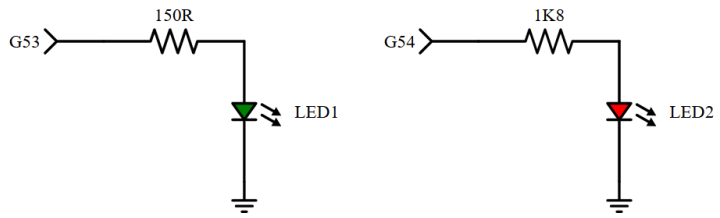


Figure 6: The Dual LED Module Schematic

There are two independent circuits inside the Dual LED Module. OK, they share the ground terminal of the port, but that's it.

The green LED is connected to GPIO pin G53. When GPIO pin G53 is set to a logic high, the ESP32 outputs 3.3V to pin G53 and the green LED lights. When GPIO pin G53 is set to a logic low, the ESP32 outputs 0V to pin G53 and the green LED turns off. A 150Ω resistor limits current to about 8.6mA to protect the LED and the microcontroller and the LED.

The red LED is connected to GPIO G54. When GPIO Pin G54 is set to a logic high, the ESP32 outputs 3.3V to pin G54 and the red LED lights. When GPIO pin G54 is set to a logic low, the ESP32 outputs 0V to pin G53 and the green LED turns off. A 1.8KΩ resistor limits current to 0.72mA to protect the LED and the microcontroller and the LED.

Connections

Figure 7: The M5Stack Grove Cable

Note that there are polarity tabs molded into the cable plug and the corresponding sockets which only lets the cable connect the proper way:

Figure 8: Grove Connector Polarizing Tabs

1. To connect the Dual LED module to the Tab5 dev board, use the 4 wire cable supplied with the module. Plug one end into the Dual LED module. Plug the other end into the Tab5 dev board.

2. Connect the Tab5 USB-C port to the computer using whatever USB cable this takes.

The Program

Our program is located in the folder `02-hello-world/code/2-blink`:

```
lessons > 02-hello-world > code > 2-blink > src > main.cpp > ...
1  /*
2    Lesson 2 - Hello World
3    by Bryan A. "CrazyUncleBurton" Thompson
4    Last Updated 3/9/2026
5  */
6
7  // Includes
8  #include <Arduino.h> // Add the Arduino library so we can use the Arduino functions like pinMode() and
9
10 // Variables
11 int greenLEDPin = 53; // Our green LED is connected to pin 53 on the M5Stack
12 int redLEDPin = 54; // Our red LED is connected to pin 54 on the M5Stack
13
```

Figure 9: Hello World Program - Part 1

```
13
14 // The stuff in setup() runs one time when the M5Stack is powered on or reset
15 void setup()
16 {
17   pinMode(greenLEDPin, OUTPUT); // set the pin mapped to the greenLEDPin to be an output pin
18   pinMode(redLEDPin, OUTPUT); // set the pin mapped to the redLEDPin to be an output pin
19 }
20
21 // The stuff in loop()runs over and over again until the M5Stack is powered off or reset
22 void loop()
23 {
24   digitalWrite(greenLEDPin, HIGH); // Turn the green LED on
25   digitalWrite(redLEDPin, LOW); // Turn the red LED off
26   delay(500); // Wait for half a second
27   digitalWrite(greenLEDPin, LOW); // Turn the green LED off
28   digitalWrite(redLEDPin, HIGH); // Turn the red LED on
29   delay(500); // Wait for half a second
30 }
31
```

Figure 10: Hello World Program - Part 2

Lines 1-5 are a block comment that lets the user know who to yell at when this doesn't work.

Line 8 is an "include". This is a library / module / shared code which adds some Arduino-style commands. Specifically, we want the **pinMode()** command which lets us declare the GPIO pin to be an output, and the **digitalWrite()** command which will turn GPIO pins on and off.

Lines 10 and 11 create global variables of the integer type. The variables

are named greenLEDPin, which we set to GPIO 53, and redLEDPin, which we set to GPIO Pin 54.

Lines 15-19 form the setup() function. This is standard Arduino form, and includes all the stuff that only needs to happen once at the time of microcontroller turn-on. The pinMode() commands make the pins referenced by the variables **greenLEDPin** and **redLEDPin** to be digital outputs.

Digital output pins can output two levels:

- Digital logic level 1, or High, which makes that output pin +3.3V
- Digital logic level 0, or Low, which makes that output pin 0V.

Lines 22-30 for the loop() function. This is standard Arduino form, and includes the stuff that runs in order from top to bottom. When it gets to the end of the list of stuff in the loop function, it loops and runs that stuff again.

Important Point: Embedded programs are designed to never end - they run as long as the device is turned on!

Lines 24 and 25 sets **greenLEDPin** to **High**, which turns the green LED on, and sets the **redLEDPin** to **Low**, which turns the red LED off.

Lines 26 is a delay. It waits 500 milliseconds (one-half second) before proceeding.

Lines 27 and 28 sets **greenLEDPin** to **Low**, which turns the green LED off, and sets the **redLEDPin** to **High**, which turns the red LED on.

Lines 26 is a delay. It waits 500 milliseconds (one-half second) before proceding.

Compiling and Uploading the Program

The microcontroller has *bootloader* code running, which helps VS Code to upload programs to the microcontroller. This code is preloaded on the Tab5 at the factory, and remains in place when programs are uploaded to the microcontroller.

1. See the upload port icon at the bottom of the screen. Ensure the upload port is set to Auto.

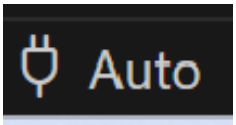


Figure 11: PlatformIO: Upload Port

2. To compile the program and upload it to the microcontroller, click on the **PlatformIO: Build** icon at the bottom of the screen:

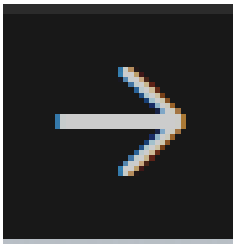


Figure 12: PlatformIO: Upload

3. Make sure the **Switch PlatformIO Project Environment** icon at the bottom of the screen is set to **2-blink**. This will ensure that we compile the correct code (there are many individual programs in the Microcontroller Training folder):

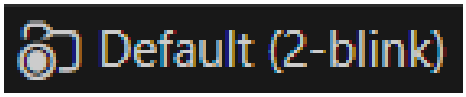
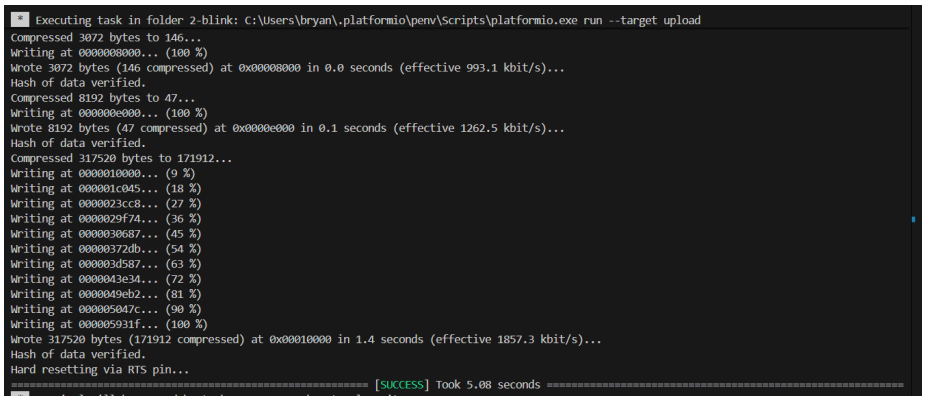


Figure 13: Switch PlatformIO Project Environment

4. The results of the compile and upload process are presented in the terminal window at the bottom of the screen:



```
Executing task in folder 2-blink: C:\Users\bryan\.platformio\penv\Scripts\platformio.exe run --target upload
Compressed 3072 bytes to 146...
Writing at 0000008000... (100 %)
Wrote 3072 bytes (146 compressed) at 0x00008000 in 0.0 seconds (effective 993.1 kbit/s)...
Hash of data verified.
Compressed 8192 bytes to 47...
Writing at 000000e000... (100 %)
Wrote 8192 bytes (47 compressed) at 0x0000e000 in 0.1 seconds (effective 1262.5 kbit/s)...
Hash of data verified.
Compressed 317520 bytes to 171912...
Writing at 0000010000... (9 %)
Writing at 000001c045... (18 %)
Writing at 0000023cc8... (27 %)
Writing at 0000029f74... (36 %)
Writing at 0000030687... (45 %)
Writing at 00000372db... (54 %)
Writing at 000003d587... (63 %)
Writing at 0000043e34... (72 %)
Writing at 0000049eb2... (81 %)
Writing at 000005047c... (90 %)
Writing at 000005931f... (100 %)
Wrote 317520 bytes (171912 compressed) at 0x00010000 in 1.4 seconds (effective 1857.3 kbit/s)...
Hash of data verified.
Hard resetting via RTS pin...
========================================================== [SUCCESS] Took 5.08 seconds ==========================================================
```

Figure 14: Program Compiled and Uploaded Successfully

The compiled program is stored in FLASH memory on the microcontroller. FLASH memory is persistent - it retains the program even after the power has been isconnected!

Running the Program

When the program uploads and the microcontroller reboots, we should see the red and green LEDs alternating on and off every 1/2 second.

Conclusion

This completes the the Getting Started / Hello World lesson. In the next lesson, we will introduce the LCD screen.